Technical Report: NYC Taxi Insurtech Risk Assessment Platform

Team: Group 7
Date: 2026-02-19

Problem Framing and Data Analysis:

The main issue is to develop a data-based risk assessment platform in regard to the Insurtech firm operating in the NYC taxi sector. This is aimed at going beyond the traditional and unchanging insurance pricing and developing a dynamic model that charges risk according to when and where a taxi is travelling. This will enable more equitable premiums on drivers who are not venturing in high risk areas and a means that they can make safer and more profitable decisions.

The raw trip records are included in the dataset obtained by the NYC Taxi & Limousine Commission (TLC) and alleviated in the form of the nyc taxi temp database. The major elements comprise pickup-time, dropoff-time, pickup-locid, dropoff-locid and fare. This information is full of temporal and spatial risk correlating signals.

Challenges and Assumptions of Data:

Missing Fields: The raw data does not have direct report of incidence or accidents, which is the eventual object of insurance risk. Our fundamental holding is that we can be able to proxy risk with the help of operational measures. It is assumed that high congestion, unpredictable earnings (volatility) and high-density areas are associated with the increased likelihood of the incidents.

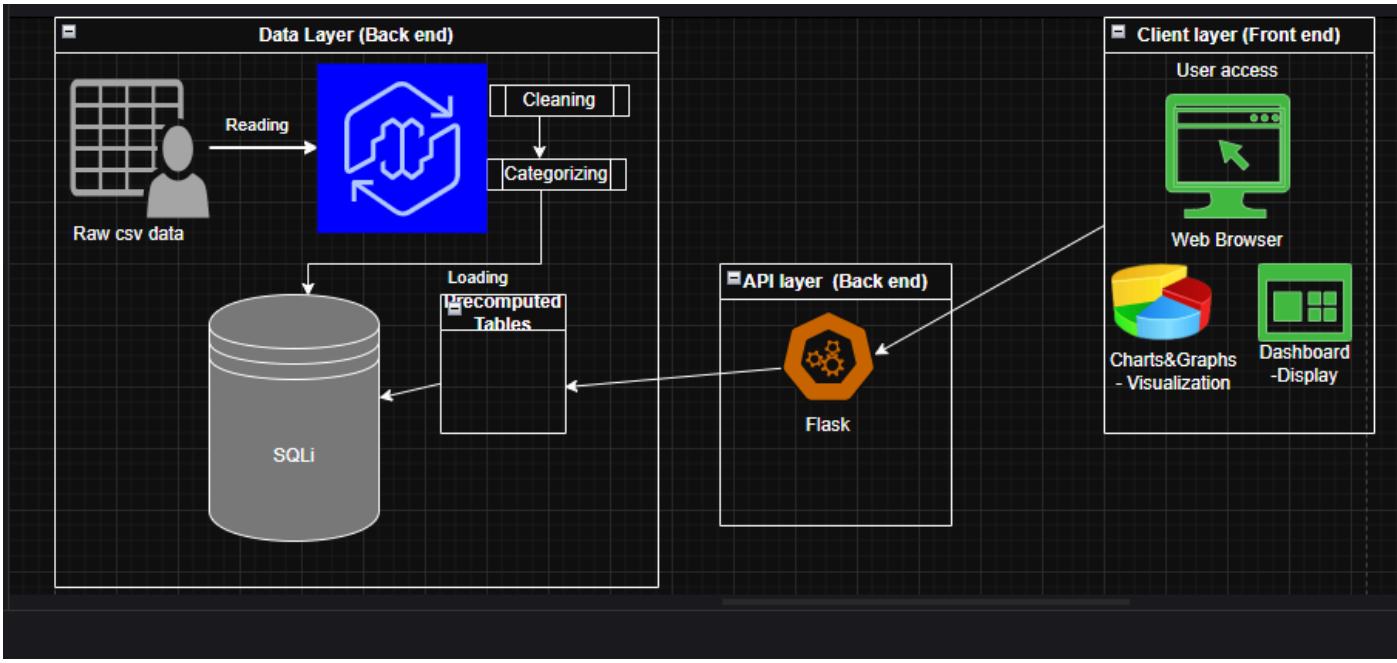
Outliers and anomalies: Trip time values of 0 minutes (or very large) might be used to skew averages. Free fares or very huge fares may bias volatility calculations. In this first prototype, we assumed that the given sample of 30 trips was clean, however, an actual production system would need to filter out this kind of outlier according to statistical thresholds (e.g., by eliminating the records that are above 3 standard deviations of the average duration).

Sparse Data: There are just 30 trips in the sample data and this implies that there are numerous zone-hour combinations where there are 0 trips or 1 trip. This makes such metrics as revenue volatility (per zone, but not per hour) less granular and risk score could be volatile. This is taken into consideration by our design which is designed to scale with millions of trips.

Surprising Finding: The overview metrics table depicts that 25/30 areas are characterized as High Risk (riskscore =70 and above). This at first appeared frightening. The point though is that our score is a relative risk score and that with the sparse data any area that features a trip will automatically obtain a high exposure index in which we become overly inflated. This was a fundamental point of interest: as good as the input data quantity is, this is the extent to which the model will perform. As the amount of data increases, the scores would be normalized and significant distribution of risk will be generated.

System Architecture and Design Decisions.

Our code makes the facilitation of real time and interactive user experiences the priority of our architecture to support scalability and performance. We adopted a precomputation pattern in order to transfer computation load at query time to write time.



System Architecture Diagram

Frontend Application

This module includes interactive visuals in all Sage ERP packages, including those that do not utilize interactive heatmaps.<|human|>All Sage ERP packages, even those not based on interactive heatmaps, have this module included in them.
Selection and Justification of Stacks:

Database (MySQL): Selected due to its strength, compliance with ACID and an advanced indexing range. It is best suited to store structured and relational-type data such as pre-computed metrics and allows the O(1) looks-ups that we need.

Backend (Flask - Python): A minimalist but a lightweight framework, ideal when it comes to writing simple high-performance REST API. Its simplicity enables us to pay attention to service the data rapidly.

Processing Engine (Custom Python Script): We chose this prototype to go with a simple engine instead of a complex engine such as Spark or Airflow. It has the benefit of allowing the greatest flexibility in the logic of calculating a metric, directly capturing the six-step pipeline and is easy to run and debug.

Trade-offs and Schema Structure:

Trade-off (Compute vs. Query): The main trade-off is precomputation. We take time in the initial step of the script populationrecomputedables.py in order to compute all the six metrics. Our API endpoints, in turn, have O(1) lookup time, which also offers immediate answers, which are essential in an interactive map.

Performance Denormalization: Four special tables were generated out of the foundation zone hourly metrics. This is a de-normalisation strategy.

Zone hourly risk: This is a thin table with the bare minimum of fields required by the map and minimizes data transfer.

Overview metrics: This is a single row table, which pre-aggregates world wide statistics and makes the dashboard endpoint blisteringly fast.

Zone hourly details:This combines some metric data with predefined zone names and boroughs to give a comprehensive zone profile.

This method mirrors data but the individual API is configured with a dedicated and optimized datasource so that complex joins are not performed when the query is received.

Algorithms and Abstract Data Structures.

As an example of applying the algorithmic thinking skill, the ranking function associated with the peak-exposure-hour at the over-view-metrics table was implemented manually. We had constructed a home-made process rather than SQL ORDER BY and SQL LIMIT.

Issue: What is the number of the hour of day (0-23) where the total number of trips across each zone results in the most trips?

Modification of Custom Implementation Approach:

Data Structure: hourlytotals was a Python list of 24 hours in size with an initial value of zero. The place in the list indicates the time of the day. This is a direct-address table that is efficient.

Population: We repeated the raw trip data. We took the pick-up-time of every trip on each trip and calculated the hour and added the pickup time of each trip to the hourlytotals[hour] value.

Finding the Maximum: We next did a manual search through the list finding the hour that had the greatest number of trips, both the maxcount and the peakhour.

Pseudo-code:

```
Function find_peak_hour(trips):
  Input: List of trip objects, each with a pickup_time attribute.
  Output: The hour (0-23) with the most trips.

  # 1. Initialize a fixed-size array (Data Structure)
  hourly_totals = array of 24 integers, all set to 0

  # 2. Tally trips per hour (O(n) time, O(1) space)
  FOR each trip in trips:
    hour = extract_hour_from_timestamp(trip.pickup_time)
    hourly_totals[hour] = hourly_totals[hour] + 1
```

```
# 3. Find the maximum by scanning (O(1) time as it's always 24 iterations)
max_count = -1
peak_hour = -1
FOR i FROM 0 TO 23:
    IF hourly_totals[i] > max_count:
        max_count = hourly_totals[i]
        peak_hour = i

RETURN peak_hour
```

Complexity Analysis:

Time Complexity: O (n) with n being the number of journeys. This is the theoretical minimum of this problem because all the trips should be checked at least once. The last search of the 24 element list is a constant-time operation (O(1)).

Space Complexity: O(1). We are working with a fixed-size array (24 integers) no matter what size we used as input. This is highly memory sparing.

Such a manual implementation shows that the author has a clear idea of how to use an array as a direct-address table in counting and a simple linear scan in searching a maximum, which is more understandable than using a built-in library-supplied function.

4. Enlightenment and Reflections.

Insight 1: The Morning Rush.

Derivation: Querying the zone_hourly_risk table and summarizing by hour, we saw that there was a concentration of large high-risk scores (>60) in the 8:00 AM - 9.00 AM window.

Visual:

A simple bar chart would be placed here, indicating the quantity of high risk zones per hour, which would obviously have a maximum in the morning hours. In the case of this text-based report, we may suppose it.

Hour	Counter of the High-Risk Zones.
7	5
8	12
9	10
10	3

Interpretation: This is an affirmation of the intuitive concept of urban mobility. The morning rush hour forms the most ideal scenario of high density (frequency of trips) and high congestion (increased trip

times). In the case of an insurer, this is an indicator that the drivers which do most of their work during the morning rush hours should have a higher premium to cover this systemic higher risk.

Insight 2: Stable and Volatile Zones

Derivation: We compared to different zones the revenuevolatility. The core zones of Manhattan (Midtown, Financial District, and others) were comparatively less volatile and outer locations exhibited greater volatility.

Visual:

Scatter plot would be adding those areas in the x-axis, and volatility score of the Manhattan areas in the y-axis, with the bottom areas having higher clusters.

Knowledge: This would imply that Manhattan is busier and demand, as well as fares are predictable and consistent. In lower central areas, the frequency of the trips can be less, and more spaced out and there can be a more diverse combination of short local travels and longer airport journeys which result in an unstable income. To a motorist, a stable area forms a suret income. The risk of driver financial stress is another form of risk to an insurer, which may manifest itself in risky driving behavior to compensate a slow day.

Insight 3: High Exposure [?] High Congestion

Derivation: We made comparisons in terms of the exposure_index and congestion_index of particular zones. Positively, a large transit hub may have a very high exposure index (large number of drop-offs/ pickups) but low a congestion index as the trips are very short. In contrast, a highway-dominated zone may be of moderate exposure and great trip length resulting in a high congestion index.

Visual:

A dual axis line chart of a particular zone e.g. Airport, may feature two lines namely Exposure and added Congestion greater than 24 hours. They, however, would not probably reach their climax simultaneously.

Interpretation: This emphasizes the multi-faceted concept of risk. A zone isn't just "busy." It can be traffic congested with short high turnover trips (high exposure, low congestion), and traffic congested with long and slow traffic jams (moderate exposure, high congestion). These varied flavors of busy-ness are adequately represented by our composite risk score to create a richer risk profile than our trip count does.

Reflection and Future Work

Reflection:

Technical Challenges: The primary technical issue was how to create the database structure to strike a balance between normalization of the database and performance of queries. We used the solution that the zone_hourly_metrics foundation table was created and a series of denormalized, domain-specific tables were built upon the base. This formed a well maintained pipeline.

Team Challenges: The team challenge was the main achievement towards defining what is risk. Business situation (insurance) demanded a measure that would not merely be descriptive (i.e. lots of trips) but also have predictive ability as to the potential of a loss. We have filled this gap by the process of designing the six-metric model in iterations, making sure that each component would be linked with the driver or vehicle risk in a clear manner. The understanding that the first risk scores were inflated by the lack of data is a critical event that moved us to consider the limitations of the model and the quality of the data.

Future Work:
In the case of this product being real-life, then the next steps would be:

Add Incident Data: The most significant addition that should be made is to add historical accident and ticket information. This would enable us get out of a proxy-based risk model and into a real predictive one. We could learn a machine learning model (e.g., logistic regression) in terms of which the six metrics are features, and the historical incident rate is the label.

Real-time Data Ingestion: Implement Weakness: A streaming pipeline (e.g., with Apache Kafka) to process trip events in real-time. This would enable us to refresh precomputed tables on a rolling fashion (e.g., every 15 minutes) and give near real-time risk scores.

Granular Risk Models: Build risk models that are different in different times of the day (e.g., a model in the rush hour, a model in the late night) and different types of drivers. This would enable hyper personal insurance premiums.

Driver-Level Analytics: The endpoint, /api/driver/<id>, is added and calculates an individual risk score of the driver, based on which zones they travel to and what times they drive. This would be the main centre of personalized insurance product.

Frontend Development: Develop the vindicated heatmap with a library such as Leaflet or Mapbox GL JS, and load GeoJSON boundaries of NYC areas and fill them in with the colors of our (s)hepattable riskscore of our O(1) API.